

XiaoZhi AI Chatbot Documentation Center | XiaoZh... > Development Gu... > MQTT + UDP Hybrid Communication Protocol Documentation

MQTT + UDP Hybrid Communication Protocol Documentation

Documentation for MQTT + UDP hybrid communication protocol organized based on code implementation, outlining how devices and servers interact through MQTT for control message transmission and UDP for audio data transmission.

1. Protocol Overview

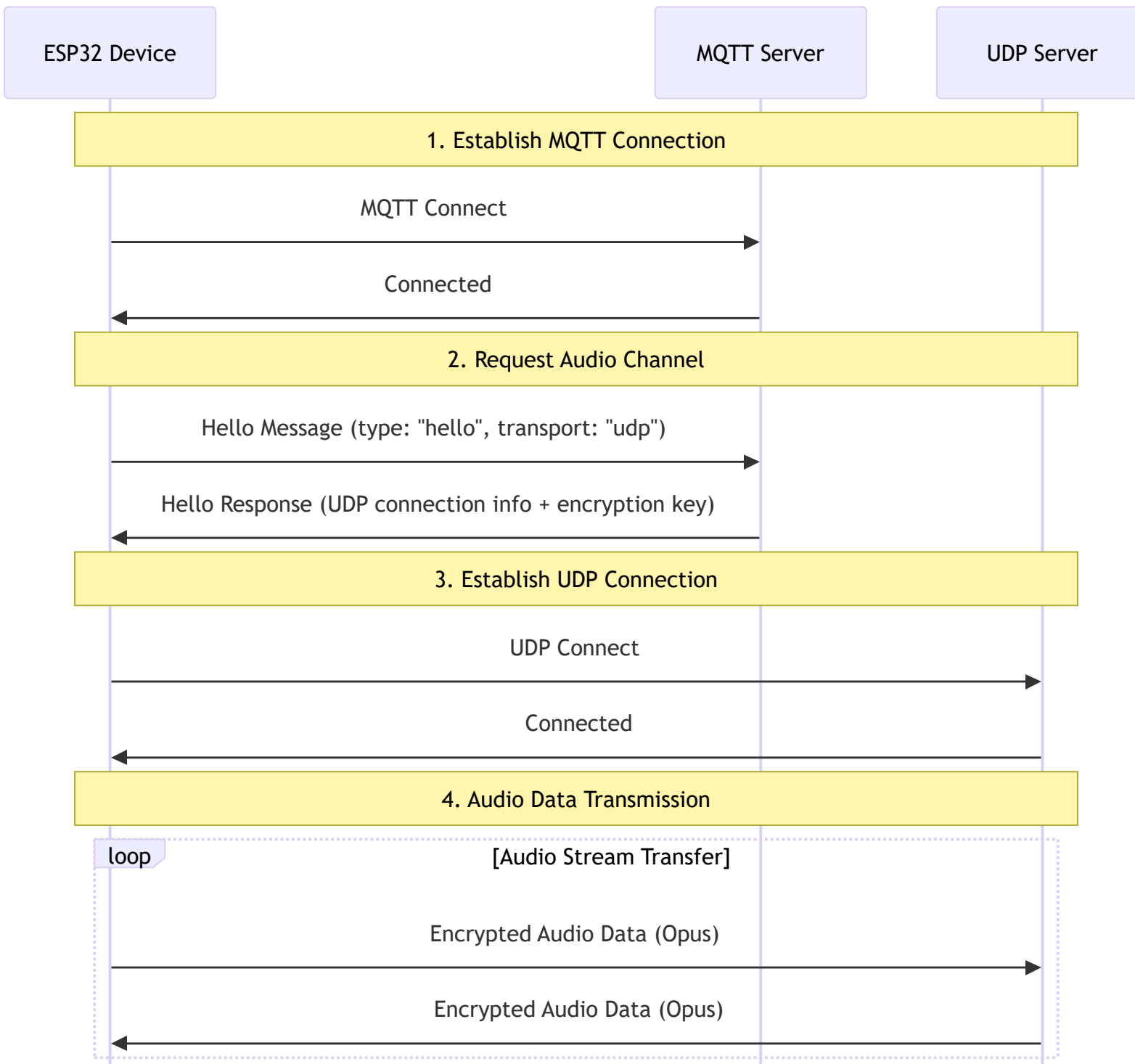
This protocol adopts hybrid transmission:

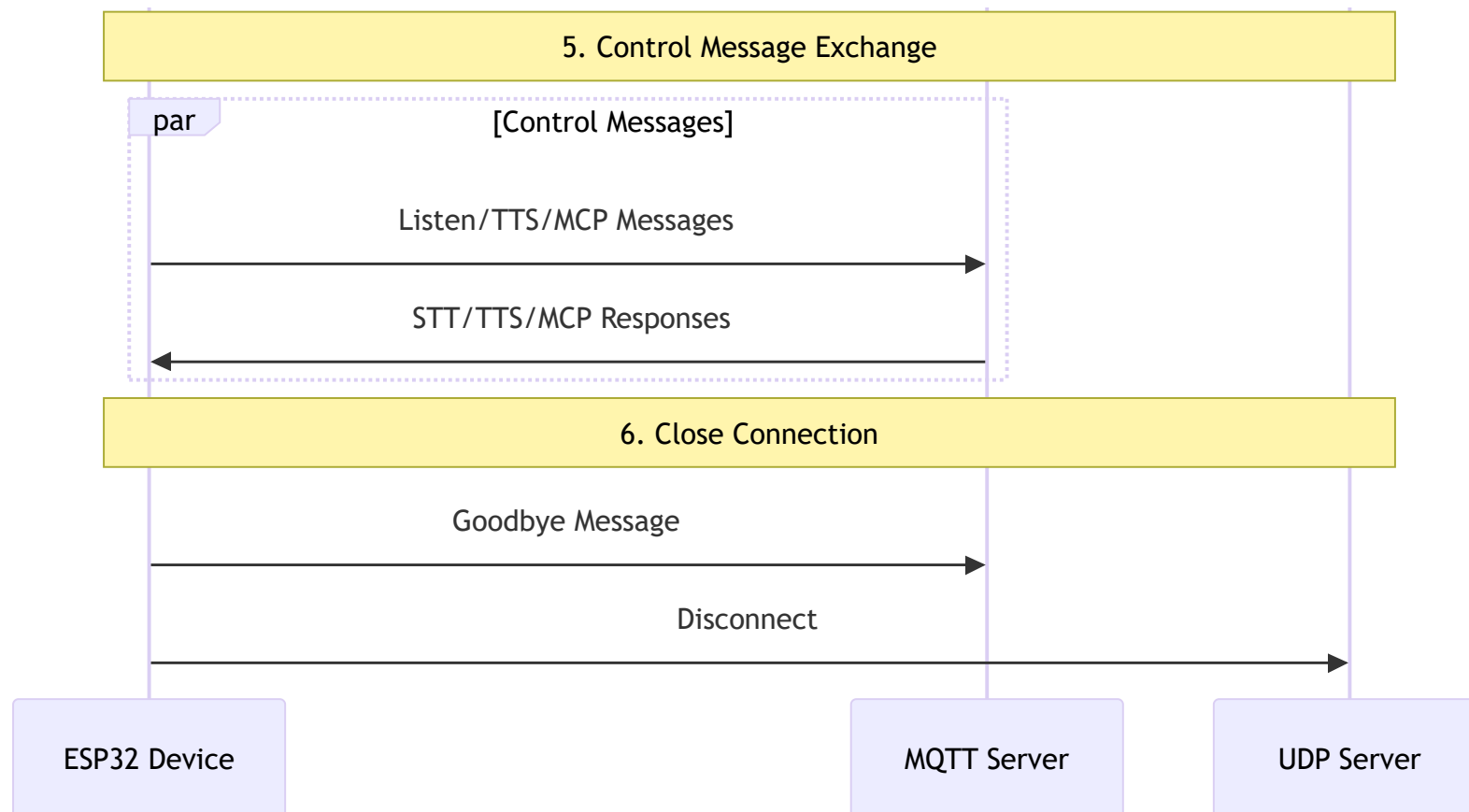
- **MQTT**: For control messages, status synchronization, JSON data exchange
- **UDP**: For real-time audio data transmission with encryption support

1.1 Protocol Features

- **Dual-channel design**: Control and data separation ensuring real-time performance
- **Encrypted transmission**: UDP audio data uses AES-CTR encryption
- **Sequence number protection**: Prevents data packet replay and disorder
- **Auto-reconnection**: Automatic reconnection when MQTT connection drops

2. Overall Flow Overview





3. MQTT Control Channel

3.1 Connection Establishment

Device connects to server via MQTT with connection parameters including:

- **Endpoint:** MQTT server address and port
- **Client ID:** Device unique identifier
- **Username/Password:** Authentication credentials

- **Keep Alive:** Heartbeat interval (default 240 seconds)

3.2 Hello Message Exchange

3.2.1 Device Sends Hello

```
{
  "type": "hello",
  "version": 3,
  "transport": "udp",
  "features": {
    "mcp": true
  },
  "audio_params": {
    "format": "opus",
    "sample_rate": 16000,
    "channels": 1,
    "frame_duration": 60
  }
}
```

3.2.2 Server Responds Hello

```
{
  "type": "hello",
  "transport": "udp",
  "session_id": "xxx",
  "audio_params": {
    "format": "opus",
    "sample_rate": 24000,
  }
}
```

```
    "channels": 1,  
    "frame_duration": 60  
  },  
  "udp": {  
    "server": "192.168.1.100",  
    "port": 8888,  
    "key": "0123456789ABCDEF0123456789ABCDEF",  
    "nonce": "0123456789ABCDEF0123456789ABCDEF"  
  }  
}
```

Field Description:

- `udp.server` : UDP server address
- `udp.port` : UDP server port
- `udp.key` : AES encryption key (hexadecimal string)
- `udp.nonce` : AES encryption nonce (hexadecimal string)

3.3 JSON Message Types

3.3.1 Device→Server

[Listen Message](#) Abort Message MCP Message Goodbye Message

Listen Message

```
{  
  "session_id": "xxx",
```

```
"type": "listen",  
"state": "start",  
"mode": "manual"  
}
```

3.3.2 Server→Device

Supported message types consistent with WebSocket protocol, including:

- **STT**: Speech recognition results
- **TTS**: Text-to-speech control
- **LLM**: Emotion expression control
- **MCP**: IoT control
- **System**: System control
- **Custom**: Custom messages (optional)

4. UDP Audio Channel

4.1 Connection Establishment

After device receives MQTT Hello response, it uses UDP connection information to establish audio channel:

1. Parse UDP server address and port
2. Parse encryption key and nonce
3. Initialize AES-CTR encryption context

4. Establish UDP connection

4.2 Audio Data Format

4.2.1 Encrypted Audio Packet Structure

```
|type 1byte|flags 1byte|payload_len 2bytes|ssrc 4bytes|timestamp 4bytes|sequence 4bytes|  
|payload payload_len bytes|
```

Field Description:

- `type` : Packet type, fixed as 0x01
- `flags` : Flag bits, currently unused
- `payload_len` : Payload length (network byte order)
- `ssrc` : Synchronization source identifier
- `timestamp` : Timestamp (network byte order)
- `sequence` : Sequence number (network byte order)
- `payload` : Encrypted Opus audio data

4.2.2 Encryption Algorithm

Uses **AES-CTR** mode encryption:

- **Key**: 128-bit, provided by server
- **Nonce**: 128-bit, provided by server

- **Counter:** Contains timestamp and sequence number information

4.3 Sequence Number Management

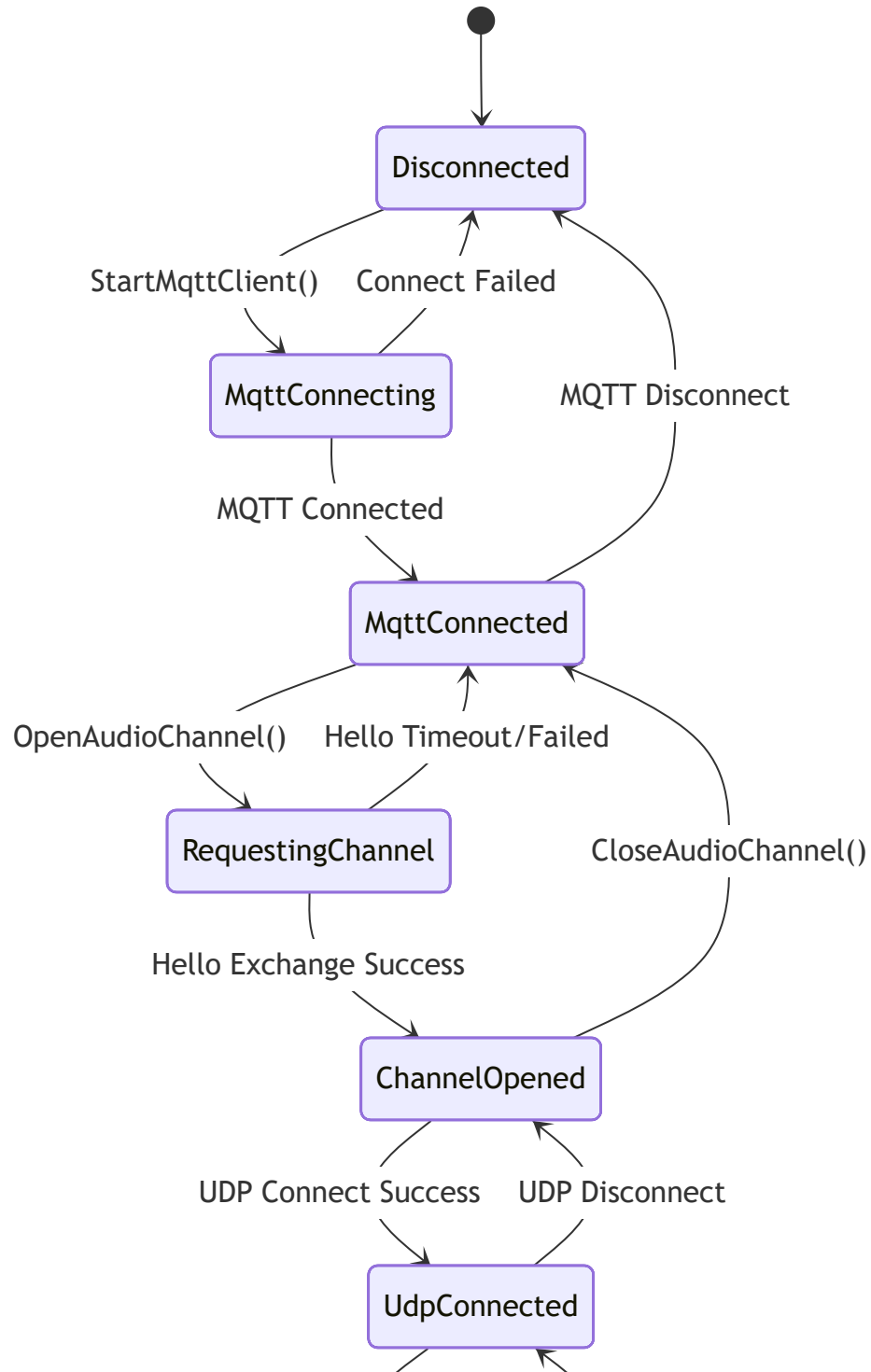
- **Sender:** `local_sequence_` monotonically increasing
- **Receiver:** `remote_sequence_` verifies continuity
- **Replay protection:** Reject packets with sequence numbers less than expected
- **Fault tolerance:** Allow minor sequence number jumps, log warnings

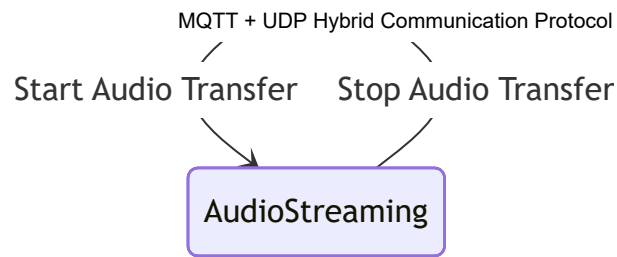
4.4 Error Handling

1. **Decryption failure:** Log error, discard packet
2. **Sequence number anomaly:** Log warning, still process packet
3. **Packet format error:** Log error, discard packet

5. State Management

5.1 Connection States





5.2 State Checking

Device determines audio channel availability through following conditions:

```
bool IsAudioChannelOpened() const {  
    return udp_ != nullptr && !error_occurred_ && !IsTimeout();  
}
```

6. Configuration Parameters

6.1 MQTT Configuration

Configuration items read from settings:

- `endpoint`: MQTT server address
- `client_id`: Client identifier
- `username`: Username
- `password`: Password
- `keepalive`: Heartbeat interval (default 240 seconds)
- `publish_topic`: Publish topic

6.2 Audio Parameters

- **Format:** Opus
- **Sample Rate:** 16000 Hz (device side) / 24000 Hz (server side)
- **Channels:** 1 (mono)
- **Frame Duration:** 60ms

7. Error Handling and Reconnection

7.1 MQTT Reconnection Mechanism

- Automatic retry on connection failure
- Support error reporting control
- Trigger cleanup process on disconnection

7.2 UDP Connection Management

- No automatic retry on connection failure
- Depends on MQTT channel for renegotiation
- Support connection status query

7.3 Timeout Handling

Base class `Protocol` provides timeout detection:

- Default timeout: 120 seconds
- Calculated based on last receive time
- Automatically marked as unavailable on timeout

8. Security Considerations

8.1 Transmission Encryption



- **MQTT**: Supports TLS/SSL encryption (port 8883)
- **UDP**: Uses AES-CTR encryption for audio data

8.2 Authentication Mechanism

- **MQTT**: Username/password authentication
- **UDP**: Key distribution via MQTT channel

8.3 Replay Attack Prevention

- Monotonically increasing sequence numbers
- Reject expired packets
- Timestamp verification

9. Performance Optimization

9.1 Concurrency Control

Protect UDP connection with mutex:

```
std::lock_guard<std::mutex> lock(channel_mutex_);
```

9.2 Memory Management

- Dynamically create/destroy network objects
- Smart pointer management for audio packets
- Timely release of encryption context

9.3 Network Optimization

- UDP connection reuse
- Packet size optimization
- Sequence number continuity checking

10. Comparison with WebSocket Protocol

Feature	MQTT + UDP	WebSocket
Control Channel	MQTT	WebSocket
Audio Channel	UDP (encrypted)	WebSocket (binary)
Real-time Performance	High (UDP)	Medium
Reliability	Medium	High
Complexity	High	Low
Encryption	AES-CTR	TLS
Firewall Friendly	Low	High

11. Deployment Recommendations

11.1 Network Environment



- Ensure UDP port accessibility
- Configure firewall rules
- Consider NAT traversal

11.2 Server Configuration

- MQTT Broker configuration

- UDP server deployment
- Key management system

11.3 Monitoring Metrics

- Connection success rate
- Audio transmission latency
- Packet loss rate
- Decryption failure rate

12. Summary

MQTT + UDP hybrid protocol achieves efficient audio-visual communication through following design:

- **Separated architecture:** Control and data channels separated, each serving its purpose
- **Encryption protection:** AES-CTR ensures secure audio data transmission
- **Sequence management:** Prevents replay attacks and data disorder
- **Auto recovery:** Supports automatic reconnection after connection drops
- **Performance optimization:** UDP transmission guarantees audio data real-time performance

This protocol is suitable for voice interaction scenarios with high real-time requirements, but requires balancing network complexity and transmission performance.

Related Documentation

- [WebSocket Communication Protocol](#) - WebSocket communication protocol details
- [MCP Protocol Documentation](#) - MCP protocol interaction flow
- [MCP Usage Guide](#) - MCP protocol usage methods

Powered by XiaoZhi.Dev

© 2025 xiaozhi.dev.  ai@xiaozhi.dev